

Lab03

Danillo Lemes(RA: 174608)

```
library(webshot2)
```

```
library(readr)  
library(dplyr)
```

Anexando pacote: 'dplyr'

Os seguintes objetos são mascarados por 'package:stats':

```
filter, lag
```

Os seguintes objetos são mascarados por 'package:base':

```
intersect, setdiff, setequal, union
```

```
library(devtools)
```

Warning: pacote 'devtools' foi compilado no R versão 4.6.1

Carregando pacotes exigidos: usethis

Warning: pacote 'usethis' foi compilado no R versão 4.6.1

```
library(ggcal)  
library(ggplot2)  
library(stringr)  
library(webshot2)
```

Warning: pacote 'webshot2' foi compilado no R versão 4.6.1

1. Importe, utilizando o pacote `readr`, cada um dos três arquivos disponíveis. Os objetos resultantes devem ser chamados `flights`, `airlines` e `airports`.

- Para o arquivo de aeroportos, importe apenas as colunas `IATA_CODE`, `CITY`, `STATE`, `LATITUDE`, `LONGITUDE`;
- Para o arquivo de vôos:
 1. Importe apenas as colunas `DESTINATION_AIRPORT` e `ARRIVAL_DELAY`;
 2. Leia apenas 1 milhão de linhas por vez;
 3. Remova vôos em que o aeroporto de destino comece com a letra 1;
 4. Remova registros em que existam pelo menos uma coluna faltante;
 5. Determine, para cada parte do arquivo, as estatísticas suficientes para a determinação do atraso médio por aeroporto de destino;
 6. Ao finalizar a leitura do arquivo, combine as estatísticas suficientes de modo a produzir a média de atraso por aeroporto.

```
processamento <- function(chunk, pos) {  
  chunk %>%  
    filter(!is.na(DESTINATION_AIRPORT), !is.na (ARRIVAL_DELAY))%>%  
    filter(!str_starts(DESTINATION_AIRPORT, "1")) %>%  
    group_by(DESTINATION_AIRPORT)%>%  
    summarise(  
      sum_delay = sum(ARRIVAL_DELAY),  
      n_flights = n(),  
      .groups = "drop"  
    )  
}  
  
chunk_stats <- read_csv_chunked("flights.csv",  
  callback = DataFrameCallback$new(processamento),  
  col_types = cols_only(  
    "DESTINATION_AIRPORT" = col_character(),  
    "ARRIVAL_DELAY" = col_double()),  
  chunk_size = 1000000  
)  
  
flights <- chunk_stats %>%  
  group_by(DESTINATION_AIRPORT) %>%  
  summarise(  
    sum_delay = sum(ARRIVAL_DELAY),  
    n_flights = n(),  
    .groups = "drop"
```

```

    Media_de_atraso = sum(sum_delay)/ sum(n_flights),
    total = sum(n_flights),
    .groups = "drop"
)

airlines <- read_csv("airlines.csv")

```

Rows: 14 Columns: 2

```

-- Column specification -----
Delimiter: ","
chr (2): IATA_CODE, AIRLINE

```

i Use ``spec()`` to retrieve the full column specification for this data.
i Specify the column types or set ``show_col_types = FALSE`` to quiet this message.

```

airports <- read_csv("airports.csv",
  col_select = c("IATA_CODE", "CITY", "STATE", "LATITUDE", "LONGITUDE"))

```

Rows: 322 Columns: 5

```

-- Column specification -----
Delimiter: ","
chr (3): IATA_CODE, CITY, STATE
dbl (2): LATITUDE, LONGITUDE

```

i Use ``spec()`` to retrieve the full column specification for this data.
i Specify the column types or set ``show_col_types = FALSE`` to quiet this message.

2. Selecione a operação apropriada *join* para incluir, na tabela `flights`, as colunas `CITY` e `STATE` do objeto `airports`. Para executar esta tarefa:

1. Identifique a coluna que é a chave na tabela `flights`; R: a coluna `DESTINATION_AIRPORT`
2. Identifique a coluna que é a chave na tabela `airports`; R: A coluna `IATA_CODE`
3. Quais são os aeroportos que estão listados em `flights`, mas estão ausentes em `airports`?
4. Apresente o comando que combine ambas as tabelas, indicando explicitamente as chaves;
5. Armazene a tabela resultante no objeto `flights`.

```
ausentes <- flights %>%
  anti_join(
    airports, by = c("DESTINATION_AIRPORT" = "IATA_CODE")
  )
head(ausentes)
```

```
# A tibble: 0 x 3
# i 3 variables: DESTINATION_AIRPORT <chr>, Media_de_atraso <dbl>, total <int>
```

```
left_join(flights, airports, by = c ("DESTINATION_AIRPORT" = "IATA_CODE"))
```

```
# A tibble: 322 x 7
  DESTINATION_AIRPORT Media_de_atraso total CITY        STATE LATITUDE LONGITUDE
  <chr>                <dbl> <int> <chr>        <chr>    <dbl>    <dbl>
1 ABE                  5.80  2220 Allentown  PA      40.7     -
75.4
2 ABI                  4.08  2224 Abilene    TX      32.4     -
99.7
3 ABQ                  5.61 18953 Albuquerque~ NM      35.0     -
107.
4 ABR                 -3.39   657 Aberdeen  SD      45.4     -
98.4
5 ABY                  8.68   864 Albany    GA      31.5     -
84.2
6 ACK                  4.91   487 Nantucket  MA      41.3     -
70.1
7 ACT                 10.6  1541 Waco       TX      31.6     -
97.2
8 ACV                  9.68  1267 Arcata/Eu~ CA      41.0     -
124.
9 ACY                 11.8  3530 Atlantic ~ NJ      39.5     -
74.6
10 ADK                 -3.19    89 Adak      AK      51.9     -
177.
# i 312 more rows
```

```
flights <- flights %>%
  left_join(airports, by = c ("DESTINATION_AIRPORT" = "IATA_CODE"))
head(flights)
```

```
# A tibble: 6 x 7
  DESTINATION_AIRPORT Media_de_atraso total CITY        STATE LATITUDE LONGITUDE
  <chr>                <dbl> <int> <chr>        <chr>    <dbl>    <dbl>
```

1 ABE	5.80	2220	Allentown	PA	40.7	-
75.4						
2 ABI	4.08	2224	Abilene	TX	32.4	-
99.7						
3 ABQ	5.61	18953	Albuquerque	NM	35.0	-
107.						
4 ABR	-3.39	657	Aberdeen	SD	45.4	-
98.4						
5 ABY	8.68	864	Albany	GA	31.5	-
84.2						
6 ACK	4.91	487	Nantucket	MA	41.3	-
70.1						

3. Quantos aeroportos cada estado possui? Apresente uma tabela ordenada de forma decrescente (no número de aeroportos).

```
tabela <-flights %>%
  filter(!is.na(STATE)) %>%
  group_by(STATE) %>%
  summarise(n_aeroportos = n(),
            .groups = "drop")%>%
  arrange(desc(n_aeroportos))

print(tabela, n = 54)
```

```
# A tibble: 54 x 2
  STATE n_aeroportos
  <chr>      <int>
1 TX          24
2 CA          22
3 AK          19
4 FL          17
5 MI          15
6 NY          14
7 CO          10
8 MN           8
9 MT           8
10 NC           8
11 ND           8
12 PA           8
13 WI           8
14 GA           7
15 IL           7
```

16	LA	7
17	VA	7
18	ID	6
19	WY	6
20	AL	5
21	HI	5
22	IA	5
23	MA	5
24	MO	5
25	MS	5
26	OH	5
27	OR	5
28	TN	5
29	UT	5
30	AR	4
31	AZ	4
32	IN	4
33	KS	4
34	KY	4
35	NM	4
36	SC	4
37	WA	4
38	NE	3
39	NJ	3
40	NV	3
41	OK	3
42	PR	3
43	SD	3
44	ME	2
45	VI	2
46	AS	1
47	CT	1
48	DE	1
49	GU	1
50	MD	1
51	NH	1
52	RI	1
53	VT	1
54	WV	1

4. Apresente um mapa representando todos os atrasos observados por aeroporto.

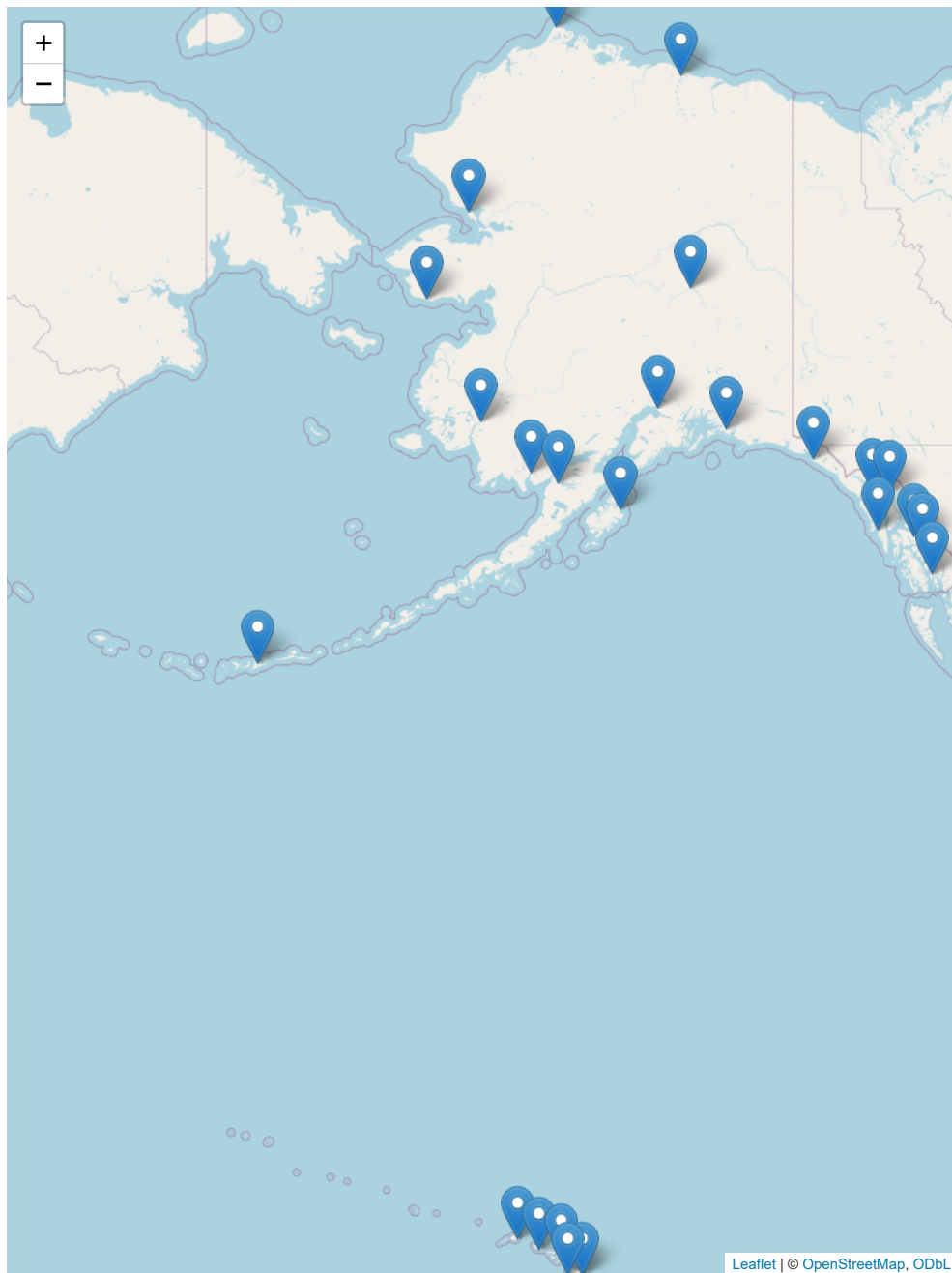
1. Carregue o pacote `leaflet`;

2. Combine os comandos `leaflet`, `addTiles` e `addMarkers` para a criação de um mapa básico;
3. Armazene o grafico em b) numa variável chamada `this`;
4. Adicione as duas linhas abaixo após a criação da variável `this` (apenas se você estiver usando Jupyter):

```
library(leaflet)
```

Warning: pacote 'leaflet' foi compilado no R versão 4.6.1

```
mapa <- flights %>%  
  filter(!is.na(LATITUDE), !is.na(LONGITUDE))  
  
this <- leaflet(data = mapa) %>%  
  addTiles() %>%  
  addMarkers(  
    lng = ~LONGITUDE,  
    lat = ~LATITUDE  
  )  
this
```



```
Sys.setenv(TZ = "America/Sao_Paulo")  
Sys.time()
```

```
[1] "2026-08-31 20:38:35 -03"
```